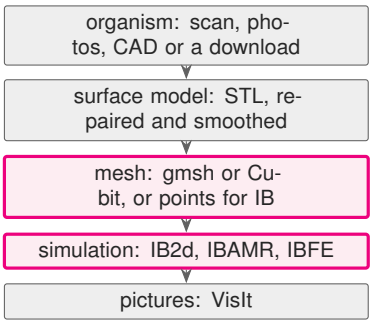
CHOOSING SOFTWARE AND BUILDING GEOMETRIES

The CFD landscape, and how an organism becomes a mesh

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part I • Chapter 1b



Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. name the main open-source and commercial CFD codes, say what each is good for, and say where IB2d and IBAMR fit;
2. explain why two codes, or two meshes, can give different answers for the same problem, and what to do about it;
3. get a 3D model of an organism or object in four ways: download it, scan it, photograph it, or draw it;
4. repair and smooth a surface so that a mesher accepts it, with tools that still exist;
5. turn the surface into a volume mesh for IBFE, or an outline into points for IB2d and IBAMR, and check it before running.

> This chapter comes from the 2024 AIMEC–NSF Cassiopea workshop (“Software for Computational Fluid Dynamics, Part 1”) and from the geometry notes of the 2018–2022 courses. Chapter 18 goes into the meshers in detail; this chapter is the map.

WHY SIMULATE AT ALL?

Experiments show you the organism; simulations show you the whole flow and the forces on every part, and let you change things that nature does not:

- the pressure and velocity inside a heart tube or around a wing, where no probe fits;
- the stiffness, shape, size or speed of the organism, one at a time;
- the Reynolds number, to ask what would happen if the animal were larger or the fluid thicker.

Two examples from the workshop: mosquito flight simulated from measured wing motion (Bomphrey et al. 2017), and jellyfish swimming driven by a neuromechanical model of the bell (Hoover et al. 2021).

> A simulation is a model. Its value lies in the question it answers and in the checks made along the way: does the answer change with the mesh, the time step, the box size, the boundary conditions? Chapters 8a, 16 and 4d are about those checks.

An equation you can solve by hand describes a sphere, not a jellyfish: detailed models need numerical simulation.

WHO DOES CFD NOW

Biologists, with the codes in this chapter:

- Eric Tytell (Tufts): swimming fish, and how the body and the flow push on each other.
- Lindsay Waldrop (Chapman): flows around the appendages of small crustaceans, in IB2d and IBAMR.
- Jake Socha (Virginia Tech): gliding snakes, with the Barba group's immersed boundary codes.

Undergraduates, in one semester:

- MATH 481 projects at the University of Arizona: for example, the flow through an embryonic heart built from a scan.
- An honors thesis at UNC on the flow around a scanned horseshoe crab, with IBFE.

> The barrier is no longer the mathematics or the computer. It is knowing which tool to pick and how to get your organism into it, which is what the rest of this chapter is about.

OPEN-SOURCE SOFTWARE

Code	2D/3D	Language, platforms	Re (rule of thumb)	Good for
IB2d	2D	MATLAB or Python; Linux, macOS, Windows	0.01–1000	learning the IB method; elastic, muscle-driven and porous boundaries; everything on a laptop (Part II)
IBAMR	2D, 3D	C++; Linux, macOS (Windows through WSL2)	0–5000	elastic and rigid immersed bodies on adaptive grids; finite element solids (IBFE); clusters (Parts III–IV)
cuIBM, PetIBM	2D (cuIBM), 2D and 3D (PetIBM)	C++ with GPUs; Linux	0–5000	rigid immersed bodies; PetIBM is the maintained successor of cuIBM (Barba group)
OpenFOAM	2D, 3D	C++; Linux (macOS and Windows through containers or WSL2)	all, with the right solver	general-purpose finite volume CFD: pipes, turbulence, heat, multiphase; moving rigid bodies; a large user community

- The Reynolds-number ranges are the workshop's rules of thumb for what each code resolves comfortably: at higher Re the boundary layers get thin and the grid cost grows.
- IB2d and IBAMR share the same structure files, so a model built for one runs in the other (chapters 4a and 6).

IB2d: https://github.com/nickabattista/IB2d. IBAMR: https://ibamr.github.io. PetIBM: https://github.com/barbagroup/PetIBM. OpenFOAM: https://openfoam.org.

COMMERCIAL SOFTWARE

Software	Platforms, cost	What it is
COMSOL Multiphysics	Windows, macOS, Linux; license	finite elements for coupled physics (flow, heat, chemistry, structures) with a graphical workflow; a good match for complex geometries at low to moderate Re
ANSYS Fluent (now part of Synopsys)	Windows, Linux; license, free student version	finite volume CFD with meshing (Workbench); the Spring 2022 CFD tutorials used it with scanned geometries
Autodesk CFD	Windows; education license	CFD inside the Autodesk CAD family
SimFlow	Windows, Linux; free limited version	a graphical front end for OpenFOAM

- These are not taught in these notes (chapter 0.1), but the geometry pipeline below is the same for all of them.
- Commercial meshers and post-processors are often the part people miss most when they move to open-source codes; gmsh and VisIt cover that ground here.

COMSOL: https://www.comsol.com. ANSYS: https://www.ansys.com. Autodesk CFD: https://www.autodesk.com/products/cfd. SimFlow: https://sim-flow.com.

WHICH ONE?

You want to . . .	Start with
learn how the IB method works, on a laptop	IB2d (Part II)
an elastic 2D organism, muscles, springs, a few hours per run	IB2d, then IBAMR for speed
3D, elastic solids, a cluster	IBAMR with IBFE (Parts III–IV)
a rigid, complicated shape at high Re	OpenFOAM, or a commercial code
many organisms in a given flow	Planktos on top of any of them (Part VI)
coupled heat, chemistry or electrics	COMSOL, or IBAMR's advection–diffusion (ch. 24)

> Three questions decide most cases: Is the body elastic? Is it 2D or 3D? How thin are the boundary layers (how high is Re)? Elastic and 2D: IB2d. Elastic and 3D: IBFE. Rigid and high Re: a body-fitted finite volume code.

Whatever you choose, the organism's geometry has to be built, cleaned and meshed first. That part is the same everywhere.

DOES ALL SOFTWARE GIVE THE SAME ANSWER?

Mesnard and Barba (2017) tried to reproduce their own published result, the lift on the cross-section of a gliding snake at Re $= 2000$ and $35°$, with four codes: cuIBM, PetIBM, IcoFOAM (OpenFOAM) and IBAMR.

- The codes did not agree at first, and it took three years to understand why.
- Causes found: the outflow boundary condition and the box size; one bad triangle in a mesh, which made spurious vorticity; changes in the linear-algebra libraries between versions; how long the wake needs to become statistically steady.
- With those fixed, the results agreed, and the paper tells you exactly what was run.

> Lessons for every simulation in these notes: record the versions (chapter 0.2), check the box and the boundaries (chapter 7), check the mesh (this chapter and 18), and run long enough and fine enough to know what is converged (chapter 16).

O. Mesnard and L. A. Barba, *Reproducible and replicable computational fluid dynamics: it's harder than you think*, Computing in Science & Engineering 19(4), 2017 (the arXiv preprint's title: "Reproducible and replicable CFD: it's harder than you think"); https://arxiv.org/abs/1605.04339.

MESH SIZE MATTERS

The workshop deck showed the same snake wake on grids from 128^2 to 4096^2 cells (from the Barba group's flying-snake studies):

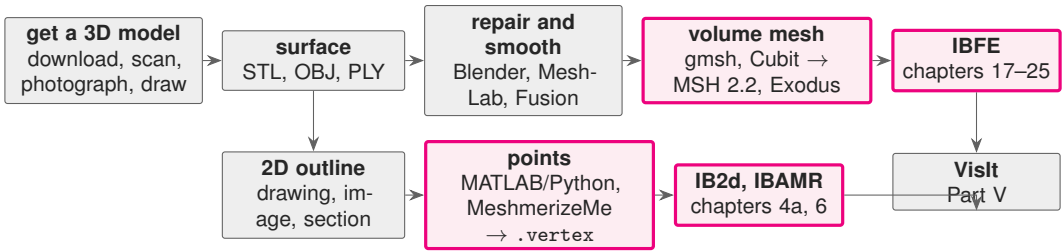
- on the coarsest grids the wake is wrong in kind, not only in detail: the vortices are smeared and shed at the wrong times;
- the picture stops changing between the two finest grids, which is the sign that the result is converged;
- the forces converge later than the pictures.

The same holds for the time step and for the number of points on an immersed boundary (chapter 5: about two points per grid cell).

> The rule: refine the grid (and the time step) until the quantity you care about stops changing, and report that quantity on at least two grids. Chapter 16 and the IB2d lab in chapter 4d show how.

Adaptive refinement (IBAMR) puts the fine cells only where the boundary and the vortices are, which is why 3D is affordable at all (chapter 5).

FROM AN ORGANISM TO A SIMULATION



- A 3D organism goes through the top row: a surface, cleaned, then filled with tetrahedra or hexahedra for IBFE.
- A 2D model needs only an outline and points along it, at about half a grid cell apart, for IB2d or IBAMR's IB method.

FOUR WAYS TO GET A 3D MODEL

Way	How	Good	Not so good
Download	repositories of scanned specimens and drawn objects (next two slides)	minutes; museum CT scans are excellent	licenses; no units in STL; shapes may be artistic rather than accurate
Scan	structured-light or laser scanner, phone LiDAR app, micro-CT	real shape and size	small, shiny, wet or translucent organisms scan badly; holes and noise
Photograph	photogrammetry: 50–100 overlapping photos reconstructed into a surface	a camera is enough; works outdoors	needs care with light and overlap; no scale unless you add one
Draw	CAD: Tinkercad, Autodesk Fusion, FreeCAD, Blender, SolidWorks	clean, parametric, in real units; easy to vary	idealized; takes time to learn

> For a first simulation, draw a simplified organism in CAD. A clean, smooth shape in real units meshes in minutes; a raw scan can take days of repair, and the simulation will not resolve its finest details anyway.

DOWNLOADING: GENERAL REPOSITORIES

Sites with STL, OBJ and PLY files of everything from gears to animals, most made for 3D printing:

- Thingiverse https://www.thingiverse.com
- Thangs https://thangs.com
- YouMagine https://youmagine.com
- Pinshape https://pinshape.com
- Sketchfab https://sketchfab.com
- Yeggi (a search engine) https://www.yeggi.com
- Libre3D https://libre3d.com
- GrabCAD (engineering parts; account needed) https://grabcad.com

> Check three things before you use a file: its license (many are Creative Commons, some non-commercial, some not for redistribution; these notes do not ship purchased models); its units, since STL has none, so measure a known length after import; and its accuracy, since a printable model is often simplified or stylized.

Links checked in October 2026; sites come and go.

DOWNLOADING: SCIENTIFIC REPOSITORIES

Repository	What it holds	Link
NIH 3D (formerly the NIH 3D Print Exchange)	anatomy, prostheses, proteins (“molecule of the month”), a heart library; scientifically accurate STL files	https://3d.nih.gov
MorphoSource	CT and surface scans of museum specimens worldwide; registration, and permission for some	https://www.morphosource.org
DigiMorph	high-resolution X-ray CT of biological specimens (University of Texas)	http://www.digimorph.org
Phenome10K	CT and surface scans of biological and palaeontological specimens, for academic use	https://www.phenome10k.org
GB3D Type Fossils	type specimens of British macrofossils	https://www.3d-fossils.ac.uk
Smithsonian 3D	scanned objects from the Smithsonian collections	https://3d.si.edu
UMORF	fossils from the University of Michigan Museum of Paleontology	https://umorf.ummp.lsa.umich.edu

- Many of these give you a CT image stack, not a surface. Segment it (mark the organism in the slices) in 3D Slicer, which is free, and export the surface as STL. https://www.slicer.org
- Scanned specimens come with their units; keep them when you export.

SCANNING

- Structured-light and laser scanners (makerspaces have them; MATH 762 used the UNC makerspace in 2019): sub-millimeter accuracy for objects from a few centimeters up; export OBJ, PLY or STL.
- Phone apps with LiDAR or photogrammetry (3D Scanner App, Polycam, Scaniverse): fast, for decimeter-sized objects, with rougher surfaces.
- Micro-CT for small organisms and for anything hollow or soft: a stack of slices, segmented into a surface (3D Slicer).

> Scan with a ruler or a calibration object in view, so that the scale can be checked. Shiny, dark, wet or translucent surfaces confuse optical scanners: dust them with powder, or use CT.

SCANNING: WHAT YOU GET BACK

- The output is a raw surface: expect holes where the scanner could not see, floating fragments, duplicated and noisy patches, and a few million triangles. The repair step (slides 18–20) is not optional.
- Scanners export OBJ or PLY (with colors) or STL; CT gives DICOM slices that 3D Slicer segments into a surface and exports as STL.
- Keep the scanner's units when you export, and write them down: STL files do not carry them.
- A scan of a live or preserved animal is a snapshot of one posture; for a moving organism (a swimming jellyfish, a flapping wing) you will still prescribe the motion in the simulation (chapters 9a, 19 and 21).

3D Scanner App (iPhone and iPad): https://apps.apple.com/us/app/3d-scanner-app/id1419913995. Polycam: https://poly.cam. Scaniverse: https://scaniverse.com. 3D Slicer: https://www.slicer.org.

PHOTOGRAMMETRY

Many overlapping photographs are matched into a point cloud and then a surface.

- Software: Autodesk ReCap Photo (education access; https://www.autodesk.com/products/recap), Meshroom (free; https://alicevision.org), RealityCapture, Polycam, and the phone apps above.
- From the 2022 course tutorial: 50 to 100 photos with 60–80% overlap, at two or three heights all around the object; manual focus and fixed exposure; even, diffuse light; a matte background that does not move.
- Afterwards: slice away the background and the table, fill the holes, then smooth and reduce (slides 18–20).

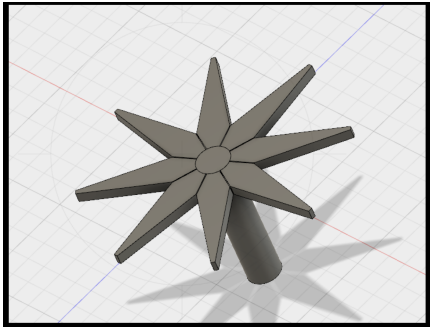
> Photogrammetry gives shape, not size. Put a ruler in the scene, or measure one dimension of the real object, and scale the model to it before you mesh.

A turntable with the camera fixed also works, if the background is plain and the light does not change.

DRAWING IT: CAD

- Tinkercad (in the browser): the place to start; simple solids combined.
- Autodesk Fusion (free for education): parametric CAD, plus a mesh workspace that repairs, reduces and converts meshes to solids. Tutorials: https://www.autodesk.com/education.
- FreeCAD and Blender (free): FreeCAD for parametric parts, Blender for sculpting organic shapes and for editing meshes.
- SolidWorks: if your institution has a license.

From the 2018 two-coral lecture: draw in real units (meters are recommended, to match the input file), and place the object where it will sit in the fluid box, so that no coordinate mapping is needed later.



The coral polyp model drawn in Fusion 360 for the 2018 two-coral IBFE example (chapter 21).

FILE FORMATS YOU WILL MEET

Format	What it is	Where it goes
STL	triangles only, no units, no connectivity (binary or text)	out of CAD and scanners; into meshers and repair tools
OBJ, PLY	triangles with normals, colors or texture; PLY is common for scans	scanners and photogrammetry; convert to STL for meshing
STEP, IGES	exact CAD solids (surfaces, not triangles)	between CAD programs; gmsh and Cubit can mesh them directly
MSH	gmsh's own format; version 2.2 text is what libMesh reads best	the mesh file of IBFE examples 05 and 10
Exodus (`.e`, `.ex2`)	the Sandia finite element format; Cubit writes it, IBFE reads and writes it	IBFE/08's meshes; every IBFE run's `output.ex2`
`.poly`	TetGen and Triangle input: points and boundary segments	2D and 3D meshes made from an outline (chapter 18)
SVG	2D vector drawing	MeshmerizeMe's input for IB2d and IBAMR points

> libMesh reads text files. A binary MSH stops IBAMR with `Unknown data format for mesh in Gmsh reader`, and an MSH 4 binary file is the gmsh default: save with `-format msh22` or choose ASCII in the export dialog (IBFE/05's README).

WHY A SURFACE NEEDS REPAIR

A mesher wants a closed, manifold surface: every edge shared by exactly two triangles, no triangle crossing another, inside and outside well defined. Scans and downloads have:

- holes and missing patches;
- floating fragments and duplicated vertices;
- self-intersections and inside-out normals;
- noise, and features far smaller than the grid will resolve;
- millions of triangles, more than any mesher needs.

> Bad surfaces make bad volume meshes, and bad meshes make bad simulations: very flat “sliver” tetrahedra deform wildly (IBFE/10’s README shows J straying from 0.2 to 2.7 in them), and tiny elements force a tiny time step.

Aim for a surface whose triangles are about the size of the fluid grid cell Δx at the organism: finer features are invisible to the simulation and only cost time.

TOOLS FOR REPAIR AND SMOOTHING

Tool	Operations	Status (October 2026)
Autodesk Meshmixer	Inspector (auto-repair), Make Solid, Smooth, Reduce, Remesh; the tool of the 2018–2024 courses	no longer developed; its web site now redirects to an Autodesk app listing. Keep a copy if you have one; otherwise use the tools below
Blender	3D-Print Toolbox (checks), Remesh modifier (voxel size $\approx \Delta x$), Smooth, Decimate, Solidify, Boolean	free, https://www.blender.org
MeshLab	Close Holes, Remove Duplicates, Screened Poisson surface, Laplacian and Taubin smoothing, Quadric decimation	free, https://www.meshlab.net
Autodesk Fusion	Mesh workspace: Repair, Reduce, Smooth, Convert Mesh to a solid body	free for education
gmsh	Reclassify and Remesh an STL (Mesh $\rightarrow$ Reclassify 2D)	free, https://gmsh.info

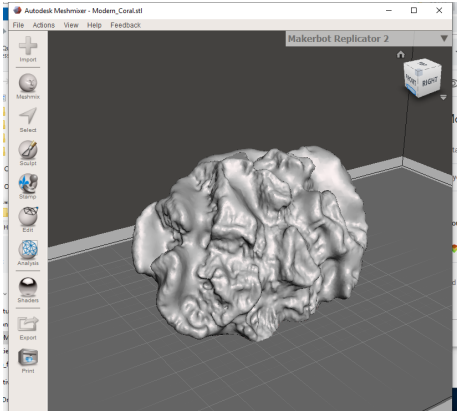
- The operations are the same in every tool: repair (close, remove, fix normals), remesh to a uniform triangle size, smooth, reduce the count, export STL. Trelis and Bolt, the csimsoft meshers of the 2018 notes, became Coreform Cubit (next slides).

THE SMOOTHING RECIPE, OLD AND NEW

As done in Meshmixer (legacy, 2018–2024)

1. Import the STL; Analysis → Inspector, auto-repair all.
2. Edit → Make Solid: a closed surface, with the “solid accuracy” set so that the cell size is about Δx.
3. Sculpt → Smooth (brush) or Deform → Smooth (whole object).
4. Edit → Reduce to the triangle count you want.
5. Export STL (binary is fine for meshers; text for inspection).

The same in Blender: a Remesh modifier (Voxel, size $\approx \Delta x$), applied; Smooth (or Corrective Smooth); Decimate; File → Export → STL.



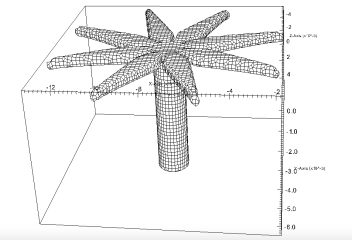
The scanned coral in Meshmixer after Make Solid and smoothing (2024 workshop). The original scan had holes and a rough surface.

FROM A SURFACE TO A VOLUME MESH

- gmsh (free): tetrahedra from a closed STL, a `.geo` script or a STEP file. Text format 2.2 is what IBFE reads.
- Coreform Cubit: tetrahedra *and* hexahedra, Exodus output, a free Learn edition with a limit on mesh size; the successor of Trelis and Bolt. SALOME, TetGen (`.poly`) and Triangle (2D): chapter 18.

```
// coral.geo: mesh a repaired STL with gmsh
Merge "coral_smooth.stl";
Surface Loop(1) = {1};  Volume(1) = {1};
Physical Volume("coral") = {1};
Mesh.CharacteristicLengthMax = 0.0005;  // about dx
```

```
gmsh -3 -format msh22 coral.geo -o coral.msh
```



The 2018 coral polyp meshed with hexahedra in Bolt (now Coreform Cubit), elements about one grid cell across.

Element size $\approx \Delta x$ of the finest grid level: finer costs time and tiny time steps; coarser lets fluid leak through the solid (chapter 17).

CHECKING A MESH BEFORE IBFE

1. A text file: MSH 2.2 (or MSH 4 text) or Exodus.
2. One physical volume (gmsh) or block (Cubit) holding the volume elements; once any physical group exists, gmsh saves only what is in groups (IBFE/05).
3. Elements with positive Jacobians, or IBAMR stops ("the Jacobian of each element must be positive"); gmsh's Reverse mesh fixes an inverted surface.
4. Element size about Δx; no slivers: gmsh's *Optimize 3D* or *Netgen optimize*, or remesh the surface first.
5. Units and position matching `input3d`: measure the bounding box.
6. A short run (`END_TIME` a few steps) before the long one.

> Chapter 18 shows each check in gmsh and in Cubit, and chapter 28 shows how to look at the mesh in VisIt before you run anything. IBFE/05's README lists the three most common mesh-file errors with IBAMR's exact messages.

gmsh reports element quality (Tools $\rightarrow$ Statistics); elements below 0.05 on its scale are the slivers to fix.

2D GEOMETRIES FOR IB2D AND IBAMR

The IB method needs points along the boundary, about half a grid cell apart, joined by springs, beams or target points (chapter 6). Ways to get them:

- Write them: the MATLAB and Python generators in every example (an ellipse, a line, a row of circles) are 30-line scripts; copy and change one.
- Draw them: MeshmerizeMe turns an SVG drawing (Inkscape, Illustrator) into IB2d and IBAMR files (chapter 4f; Senter et al. 2021).
- Trace them: from a photograph with Inkscape's Trace Bitmap, then MeshmerizeMe.
- Cut them: a section through a 3D model (Blender or MeshLab) gives the outline of a 2D model.

> Whatever makes the points, check the spacing against Δx (chapter 5) and look at the result before running: `test_IB2d_IBAMR_Geometry.m` in `examples/support` (and its Python version) plots the points and springs of any structure file.

MeshmerizeMe: D. M. Senter et al., Bioinspiration & Biomimetics 16:016008 (2021); https://github.com/dmsenter89/MeshmerizeMe.

VISUALIZATION SOFTWARE

Software	Cost, platforms	Notes
VisIt	free; Linux, macOS, Windows	reads IBAMR's SAMRAI dumps and Exodus files directly, and IB2d's VTK files; scriptable in Python; runs client–server on a cluster. Used throughout Part V. https://visit-dav.github.io/visit-website/
ParaView	free; Linux, macOS, Windows	reads VTK and Exodus; the most common alternative; a similar client–server mode. https://www.paraview.org
Tecplot	license; Windows, Linux, macOS	common in engineering groups. https://tecplot.com
MATLAB, Python		for numbers rather than pictures: forces, averages, convergence (chapters 11 and 33)

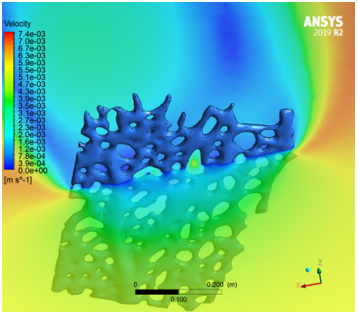
- VisIt's home moved from the LLNL web site to GitHub; the old link still redirects. Whichever tool you use, the first picture to make of a new mesh is the mesh itself, before any flow (chapter 28).

A WORKED PATH: THE SCANNED CORAL

The 2024 workshop followed one coral from a scan to a flow:

1. a scanned coral surface, with holes and noise (STL);
2. Meshmixer: Make Solid, Smooth, Reduce (previous slides);
3. a flow tank around it in a commercial code (the workshop's Parts 2 and 3, ANSYS Fluent and COMSOL: out of scope here);
4. velocity and pressure fields around and through the branches.

The same smoothed STL goes into gmsh (slide 21) and then into IBFE's flow tank (chapter 22, example IBFE/10), with tether points to hold it (chapter 19).



Velocity magnitude around the smoothed coral in a flow tank (ANSYS Fluent, 2019). The geometry work is the same for IBFE.

LEGACY TOOLS AND WHAT REPLACED THEM

In the older notes	Now (checked October 2026)
Meshmixer (2018–2024 courses)	no longer developed; Blender, MeshLab or Autodesk Fusion's mesh workspace
Trelis and Bolt (csimsoft, 2018)	Coreform Cubit, with a free Learn edition
Fusion 360	Autodesk Fusion (renamed; still free for education)
cuIBM	archived; PetIBM is the Barba group's current code
NIH 3D Print Exchange (`3dprint.nih.gov`)	NIH 3D, https://3d.nih.gov (the old address redirects)
VisIt at `wci.llnl.gov`	https://visit-dav.github.io/visit-website/ (the old address redirects)
FreeCAD at `freecadweb.org`	https://www.freecad.org
ANSYS	part of Synopsys since 2025; the products keep their names
Autodesk ReCap Photo for photogrammetry	still exists with education access; Meshroom and the phone apps are the free routes

Every link in this chapter was opened in October 2026. Some sites block automatic checks and were opened by hand.

WHAT CAN GO WRONG

Symptom	Cause and fix
The organism is 1000 times too big or too small	STL has no units; the exporter assumed millimeters. Scale the model, or set the box in `input3d` to match.
Nothing happens, or “Jacobian . . . negative” at the first step	the mesh is not where the fluid box is, or it was moved with a coordinate mapping into a wall. Check the bounding box (IBFE/10’s `TRANSLATE_X` shows the mapping).
gmsh takes hours or runs out of memory	too many surface triangles; reduce the surface to about Δx first.
“surface not closed”, “self-intersecting” from the mesher	repair the surface (slide 19); Make Solid or Remesh usually fixes both.
`Unknown data format for mesh in Gmsh reader`	a binary MSH; save as text, format 2.2 (IBFE/05).
“the Jacobian of each element must be positive”	inverted elements; reverse the surface or run `fix_orientation.py` (IBFE/05).
J far from 1 in a few elements; a tiny stable time step	slivers; Optimize 3D in gmsh, or remesh.
Fluid leaks through the solid	elements much larger than Δx; refine the mesh or coarsen the grid (chapter 17).

SUMMARY

1. IB2d (2D, laptop) and IBAMR with IBFE (3D, cluster) are the codes of these notes; OpenFOAM and the commercial codes cover rigid, high-Re flows.
2. Different codes and different meshes give different answers until the boundaries, the mesh and the run length are checked; report converged quantities.
3. A 3D model comes from a download, a scan, photographs or CAD; draw a simplified organism in real units for your first simulation.
4. Repair and smooth the surface to triangles about Δx across (Blender, MeshLab or Fusion, now that Meshmixer is gone), then mesh it with gmsh or Coreform Cubit as a text MSH 2.2 or Exodus file.
5. For 2D, write or draw the boundary points; check them before you run.

> Next, chapter 2: installing IBAMR with autoibamr. Chapter 18 returns to the meshers in detail, with the examples IBFE/05, 08 and 10.

WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The 2024 workshop deck was a tour of software and links; this chapter adds the pipeline from organism to mesh, the checks before an IBFE run, and the 2D route.
- Meshmixer, used in every geometry lecture from 2018 to 2024, is no longer developed; the recipe is kept and translated to Blender and MeshLab.
- Trelis and Bolt (2018) are now Coreform Cubit; the NIH 3D Print Exchange is NIH 3D; VisIt's home moved; ANSYS belongs to Synopsys. Dead links were replaced and every link was checked.
- The 2022 CFD tutorials' ANSYS pipeline (Fusion, Meshmixer, SpaceClaim, Workbench) and the workshop's COMSOL part are out of scope; their videos are listed in chapter 0.1.
- The 2022 "3D repositories" and "photogrammetry" tutorials are folded in here (their lists and tips), without their screenshots.

REFERENCES

- O. Mesnard and L. A. Barba. Reproducible and replicable computational fluid dynamics: it's harder than you think. *Computing in Science & Engineering* 19(4):44–55, 2017 (arXiv: "Reproducible and replicable CFD: it's harder than you think"). https://arxiv.org/abs/1605.04339
- R. J. Bomphrey, T. Nakata, N. Phillips and S. M. Walker. Smart wing rotation and trailing-edge vortices enable high frequency mosquito flight. *Nature* 544:92–95, 2017.
- A. P. Hoover, N. W. Xu, B. J. Gemmell, S. P. Colin, J. H. Costello, J. O. Dabiri and L. A. Miller. Neuromechanical wave resonance in jellyfish swimming. *PNAS* 118(11), 2021.
- D. M. Senter, D. R. Douglas, W. C. Strickland, S. G. Thomas, A. M. Talkington, L. A. Miller and N. A. Battista. A semi-automated finite difference mesh creation method for use with immersed boundary software IB2d and IBAMR. *Bioinspiration & Biomimetics* 16:016008, 2021.
- C. Geuzaine and J.-F. Remacle. Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities. *Int. J. Numer. Meth. Engng* 79:1309–1331, 2009. https://gmsh.info
- Coreform Cubit: https://coreform.com/coreform-cubit/. Blender: https://www.blender.org. MeshLab: https://www.meshlab.net. 3D Slicer: https://www.slicer.org.
- Sources: L. A. Miller, *Software for Computational Fluid Dynamics, Part 1*, AIMEC–NSF Joint Workshop for the Model Species Cassiopea, 2024; the 2018 MATH 769 lecture on 3D target points; the 2020 IBFE lecture 5 tool list; the Spring 2022 CFD tutorials on 3D repositories and photogrammetry.